

Arquitectura y API REST - Documentación Técnica

TUS DATOS AQUÍ

FECHA

Índice

1. Arquitectura de Sistemas	2
2. Modelo de Datos	2
3. API REST y Consumo (Swagger UI)	2
3.1. Operaciones Disponibles:	3
4. Arquitectura de Tiempo Real (WebSockets)	4
4.1. Gestor de Conexiones	4
4.2. Estructura del Payload (JSON)	4
4.3. Probador de WebSockets (Tester Local)	4
5. Guía de Despliegue Local	4
5.1. Requisitos Previos	5
5.2. Levantar el Backend (FastAPI)	5
5.3. Levantar el Frontend (React)	5

1. Arquitectura de Sistemas

El proyecto **Catastro Metropolitano** se basa en una arquitectura desacoplada orientada a microservicios, diseñada para alta disponibilidad y sincronización en tiempo real.

- **Frontend:** Desarrollado con React 19 y empaquetado con Vite. Utiliza Tailwind CSS v4 para la estilización. Está alojado de manera estática y distribuida globalmente en Cloudflare Pages.
- **Backend:** Desarrollado en Python 3.11 con **FastAPI 0.136** y Pydantic v2 para validación estricta de esquemas. Alojado en la plataforma Render.
- **Persistencia (Base de Datos):** Supabase (PostgreSQL) usando el cliente oficial de Python. Se garantiza la atomicidad y consistencia de los datos geoespaciales.

2. Modelo de Datos

La tabla principal en la base de datos se denomina **mapas** y cuenta con la siguiente estructura:

- **clave** (TEXT PRIMARY KEY): Identificador único del mapa (ej. sur-01).
- **nombre** (TEXT NOT NULL): Nombre legible del mapa.
- **color_tema** (TEXT): Color hexadecimal asociado al mapa.
- **width** y **height** (INTEGER): Dimensiones del canvas en píxeles.
- **config** (JSONB): Almacena toda la geometría compleja (trazos, polígonos, capas) en un formato JSON flexible.

3. API REST y Consumo (Swagger UI)

El Backend expone una interfaz interactiva de documentación generada automáticamente mediante OpenAPI/Swagger, facilitando las pruebas de integración.

Explorar API en Producción (Swagger)

API de Mapas Metropolitanos 1.0.0 OAS 3.1

/api/openapi.json

API REST para la gestión y visualización de mapas catastrales urbanos. Permite crear, leer, actualizar y eliminar mapas catastrales, incluyendo la geometría de calles, avenidas, curvas viales y cuadras.

🔌 WebSockets (Notificaciones en Tiempo Real)

Los clientes pueden suscribirse para recibir actualizaciones inmediatas.

- **Canal Global** (`ws://<host>/ws/mapas`): Recibe eventos cuando un mapa es creado, actualizado o eliminado del listado principal.
- **Canal por Mapa** (`ws://<host>/ws/mapas/{clave}`): Recibe eventos exclusivos cuando el mapa con la `clave` especificada es modificado. Ideal para refrescar el editor interactivo en pantalla.

Mapas ^

GET /api/mapas Listar Mapas ▾

POST /api/mapas Crear Mapa ▾

GET /api/mapas/{clave} Obtener Mapa ▾

PUT /api/mapas/{clave} Actualizar Mapa ▾

DELETE /api/mapas/{clave} Eliminar Mapa ▾

Schemas ^

HTTPValidationError > Expand all object

MapaConfig > Expand all object

MapaCreate > Expand all object

MapaResponse > Expand all object

MapaUpdate > Expand all object

ValidationError > Expand all object

Figura 1: Interfaz interactiva de Swagger UI donde se muestran los endpoints CRUD.

3.1. Operaciones Disponibles:

1. GET /api/mapas: Lista todos los mapas disponibles (solo retorna metadatos básicos para optimizar ancho de banda).
2. POST /api/mapas: Registra la estructura inicial de un mapa.
3. GET /api/mapas/{clave}: Obtiene el JSON completo, incluyendo la geometría.
4. PUT /api/mapas/{clave}: Actualiza las coordenadas y metadatos.
5. DELETE /api/mapas/{clave}: Elimina un mapa permanentemente de

Supabase.

4. Arquitectura de Tiempo Real (WebSockets)

Para evitar saturar la base de datos con peticiones constantes (polling), el backend implementa un sistema de notificaciones **Push** mediante WebSockets de FastAPI.

4.1. Gestor de Conexiones

El módulo `websocket_manager.py` administra los canales para garantizar el aislamiento de eventos:

- **Canales Específicos** (`ws://.../ws/mapas/{clave}`): Solo transmiten eventos relacionados a ese mapa. Garantiza la seguridad y eficiencia del ancho de banda.
- **Canal Global** (`ws://.../ws/mapas`): Notifica creación y eliminación de mapas para actualizar listados.

4.2. Estructura del Payload (JSON)

Cuando ocurre un evento, el servidor emite un JSON con el siguiente formato:

```
{
  "event": "mapa:updated",
  "data": {
    "clave": "sur",
    "mapa": { "config": {...} }
  }
}
```

4.3. Probador de WebSockets (Tester Local)

Se ha desarrollado un cliente HTML para probar los sockets de forma local, simulando ser el frontend.

5. Guía de Despliegue Local

Para ejecutar el proyecto en tu máquina para desarrollo o pruebas:



Figura 2: Consola del probador local mostrando eventos recibidos.

5.1. Requisitos Previos

- Python 3.11+
- Node.js 20+ con pnpm instalado

5.2. Levantar el Backend (FastAPI)

Clona el repositorio. Configura un archivo `.env` en la raíz con `SUPABASE_URL` y `SUPABASE_KEY`. Luego ejecuta en la terminal:

```
pip install -r api/requirements.txt
python -m api.main
```

Swagger estará disponible en `http://localhost:8000/api/docs`.

5.3. Levantar el Frontend (React)

Abre una nueva terminal, navega a la carpeta `client/` y ejecuta:

```
pnpm install
pnpm run dev
```

El frontend estará disponible en `http://localhost:5173`.